

リアルタイム音声対話システムの技術設計

Rust / Tokio / イベント駆動 / AI パイプライン

今日話すこと

1. システム全体像 - 音声の流れる経路
2. 音声の入口 - SIP/RTP からの音声受信
3. 音声認識（**ASR**） - VAD・フォールバック構成
4. インテント分類 - ルールベースで処理を振り分け
5. **LLM / TTS** - 応答生成と音声合成
6. 設計の工夫 - Rust選定・イベント駆動・Backpressure
7. 今後の展望 - RAG・MCP の適用

1. システム全体像

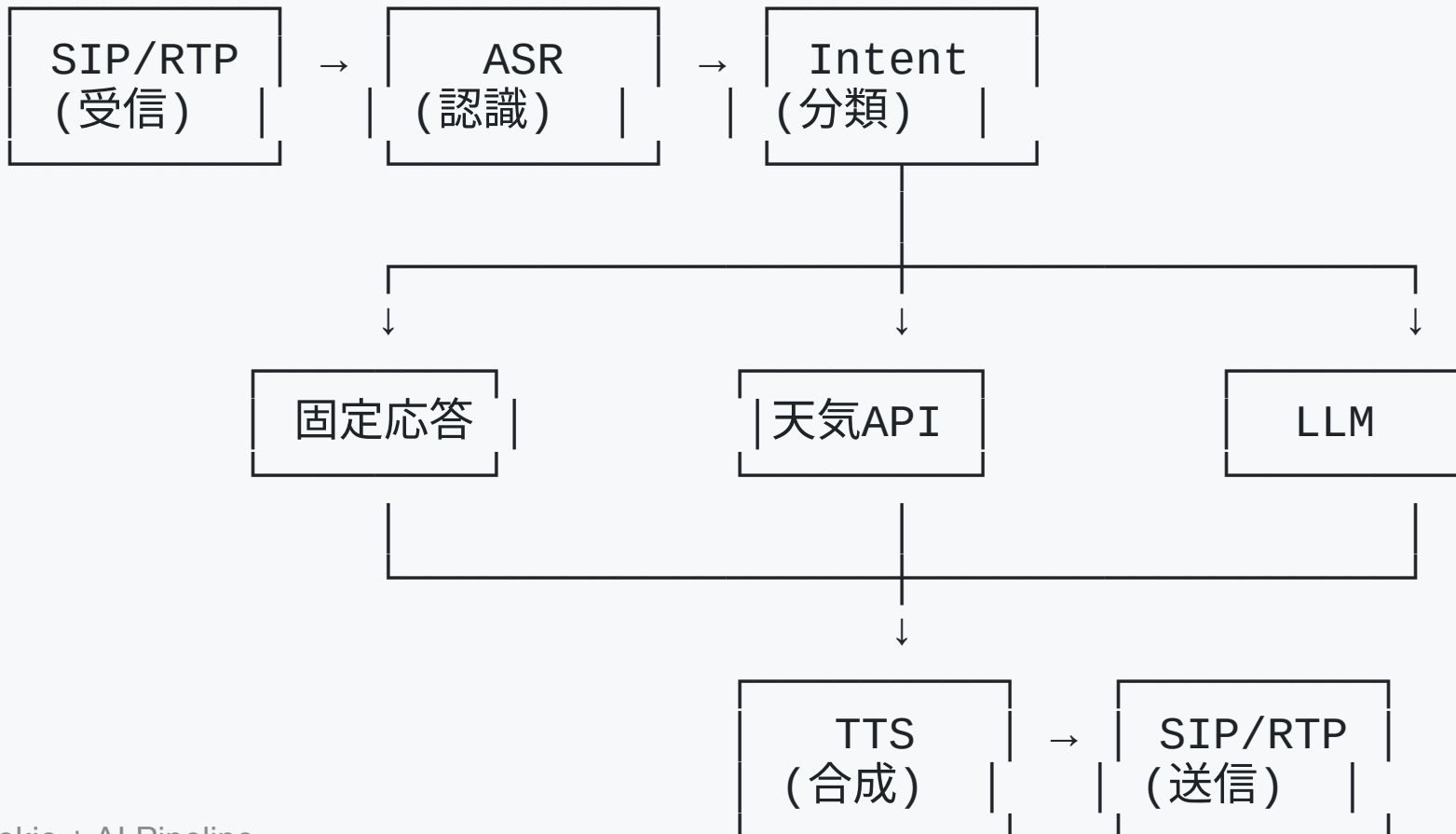


要件

- 通話中の対話遅延: **数百ms以内**
- 同時接続: 複数セッション
- 24/7 運用

データフローの全体像

電話着信



2. 音声の入口：SIP/RTP

なぜ自前実装か

観点	既存ライブラリ	自前実装
柔軟性	制約あり	完全制御
依存	重い	最小限
デバッグ	困難	容易

割り切り

- フル RFC 準拠は目指さない
- 必要な機能だけ実装（gap-analysis.md で管理）

プロトコル層の責務分離

```
graph TD; transport[transport/] --- sip[sip/]; sip --- rtp[rtp/]; rtp --- session[session/]; session --- app[app/];
```

transport/ ← 生パケット I/O のみ（意味を持たない）

sip/ ← SIP メッセージのパース・ビルド

rtp/ ← RTP パケット化、codec、ジッタバッファ

session/ ← 呼制御、状態遷移（SIP/RTPの詳細を隠蔽）

app/ ← ビジネスロジック（プロトコルを知らない）

ルール

- 上位は下位のプロトコル詳細を知らない
- SIPヘッダ、RTP SSRC などは session より上に漏らさない

3. 音声認識（ASR）の仕組み

RTP受信 20ms → μ -law decode 即時 → VAD RMS判定 → バッファ 発話単位 → WAV変換 一括 → ASR API → テキスト

ポイント

- **20ms単位** で音声パケットが届き続ける
- 「いつ発話が終わったか」を検出する必要がある（VAD）
- 終わったら一括でASRに送る（バッチ方式）

VAD（音声区間検出）

RMSエネルギーで判定

```
fn rms_energy(payload: &[u8]) -> u32 {  
    let sum: u64 = payload.iter()  
        .map(|&b| mulaw_to_linear16(b) as i64)  
        .map(|s| (s * s) as u64)  
        .sum();  
    ((sum / payload.len() as u64) as f64).sqrt() as u32  
}
```

状態遷移

Idle $\xrightarrow{\text{RMS} \geq \text{閾値}}$ InSpeech $\xrightarrow{\text{無音800ms}}$ 発話終了
|
(最大30秒) $\rightarrow$ 強制終了

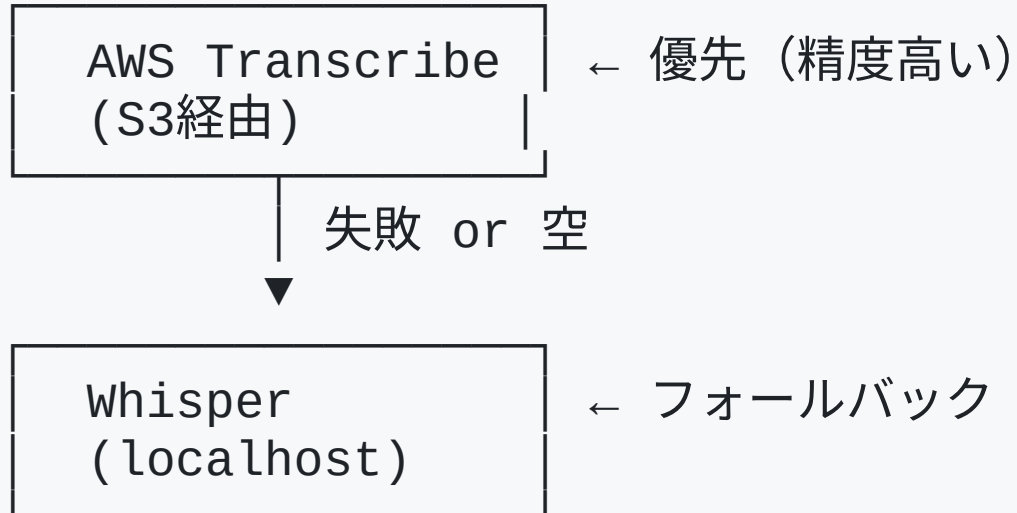
VAD パラメータ

パラメータ	値	意味
VAD_ENERGY_THRESHOLD	500	音声判定の閾値
VAD_END_SILENCE_MS	800ms	無音継続で終了判定
VAD_MIN_SPEECH_MS	300ms	最低発話長
VAD_MAX_SPEECH_MS	30,000ms	最大発話長

チューニングポイント

- 閾値低すぎ → ノイズ拾う
- 閾値高すぎ → 小声を逃す
- 無音判定短すぎ → 途中で切れる

ASRサービス構成（2段フォールバック）



なぜ2段構成か

- AWS Transcribe: 精度高い、でもたまに空を返す
- Whisper: ローカルで動く、可用性担保

ハルシネーション対策

Whisperの癖：無音でも何か返す

```
const HALLUCINATION_PATTERNS: &[&str] = &[
    "ご視聴ありがとうございました",
    "チャンネル登録",
    "高評価",
    "いいね",
];

fn is_hallucination(text: &str) -> bool {
    HALLUCINATION_PATTERNS.iter()
        .any(|p| text.contains(p))
}
```

効果

- YouTube学習データ由来の幻覚をフィルタ

4. インテント分類：ルールベースで振り分け

ユーザー発話



Intent Classifier (Ollama)
JSON形式で意図进行分类



identity	weather	transfer	general
「誰？」	「天気」	「転送」	それ以外



固定応答



天気API



転送処理



LLM応答

_intent分類の_prompt

```
const DEFAULT_INTENT_PROMPT: &str = r#"
```

あなたはボイスボットの意図分類器です。

必ずJSONのみで分類してください。

形式: {"intent": "identity|weather|transfer|general_chat",
"query": "<発話>", "params": {...}}

- identity: 「誰ですか」「名前は」
 - weather: 「天気」→ params に location
 - transfer: 「転送して」「人に代わって」
 - general_chat: それ以外
- ```
"#;
```

## ポイント

- LLMにJSON出力を強制 → パース可能
- 意図ごとに専用処理ルーティング

# なぜルールベースか

## LLMに全部任せない理由

| 全部LLM       | _intent分類  |
|-------------|------------|
| 毎回重いLLM呼び出し | 軽いLLMで振り分け |
| 応答が不安定      | 処理が決定的     |
| コスト高い       | 専用処理は安い    |

## 例：天気

「東京の天気」

- ↓ intent: weather, location: 東京
- ↓ 気象庁API呼び出し (LLM不要)
- ↓ 結果をLLMで要約 (1文)
- ↓ 「東京は晴れ、最高気温15度です」

Rust + Tokio + AI Pipeline

## 5. LLM：Gemini + Ollama フォールバック

ユーザー質問



Gemini  
gemini-2.5-flash

← 優先（精度高い）



エラー時

Ollama  
gemma3:4b

← フォールバック（ローカル）



それも失敗

「すみません、うまく答えを用意できませんでした」

## LLM フォールバックの実装

```
pub async fn handle_user_question(
 messages: Vec<ChatMessage>,
) -> Result<String> {
 // 1. まず Gemini を試す
 match call_gemini(&messages).await {
 Ok(ans) => return Ok(ans),
 Err(e) => log::error!("Gemini failed: {e:?}")
 }
 // 2. 失敗したら ollama
 match call_ollama(&messages).await {
 Ok(ans) => return Ok(ans),
 Err(e) => log::error!("Ollama failed: {e:?}")
 }
 // 3. 両方失敗したら謝罪
 Ok("すみません、うまく答えを用意できませんでした".into())
}
```

## プロンプトの外部化

prompt.local.txt があれば使う  
↓ なければ  
デフォルト：「120文字以内で回答してください」

## メリット

- コード変更なしでプロンプト調整
- 環境ごとにカスタマイズ可能
- A/Bテストがやりやすい

## TTS : VOICEVOX (ずんだもん)

テキスト



VOICEVOX (localhost:50021)

1. /audio\_query → 音声クエリ
2. /synthesis → WAV生成



WAV (16kHz, mono)



μ-law変換 → RTP送出

# なぜ VOICEVOX か

| 選択肢          | メリット          | デメリット |
|--------------|---------------|-------|
| VOICEVOX     | 無料、ローカル、日本語特化 | キャラ限定 |
| Google TTS   | 高品質、多言語       | 従量課金  |
| Amazon Polly | AWSに統一        | コスト   |

## 判断

- PoC/検証段階では VOICEVOX で十分
- 本番移行時にクラウドTTSを検討

# 6. 設計の工夫

## なぜRustか

| 要件     | Rustの強み        |
|--------|----------------|
| 低レイテンシ | GCなし、予測可能      |
| 並行処理   | Tokio による軽量タスク |
| 安全性    | メモリ安全、データ競合防止  |
| 長時間運用  | メモリリークしにくい     |

## 他の選択肢を選ばなかった理由

- Go: GCの停止時間
- C++: メモリ管理の事故リスク
- Python: レイテンシ

## 1セッション = 1タスク

```
tokio::spawn(async move {
 let session = Session::new(call_id);
 session.run().await; // 通話終了まで生存
});
```

## メリット

- セッション間で状態を共有しない → ロック競合なし
- 1セッション死んでも他に影響しない
- スケールしやすい

## イベント駆動設計

```
// session → app へのイベント
enum SessionEvent {
 CallStarted { call_id: String },
 AudioReceived { call_id: String, pcm: Vec<i16> },
 CallEnded { call_id: String },
}

// app → session への指示
enum SessionCommand {
 PlayAudio { call_id: String, pcm: Vec<i16> },
 HangUp { call_id: String },
}
```

## チャンネルで接続

```
let (event_tx, event_rx) = mpsc::channel(100);
```

# Backpressure 戦略

問題：処理が追いつかないとき何を捨てるか

| 箇所        | 戦略     | 理由         |
|-----------|--------|------------|
| RTP → ASR | 最新優先   | 遅延より鮮度     |
| TTS → RTP | 割り込み優先 | ユーザー発話で即停止 |
| LLM       | 同時実行1  | リソース保護     |

```
// 古いデータを捨てて最新を優先
while let Ok(newer) = rx.try_recv() {
 current = newer;
}
process(current).await;
```

## 相関ID (call\_id) によるトレース

```
[call_id=abc123] SIP INVITE received
[call_id=abc123] Session created
[call_id=abc123] ASR result: "こんにちは"
[call_id=abc123] LLM response: "はい、こんにちは"
[call_id=abc123] TTS started
[call_id=abc123] BYE received
```

## 効果

- 1通話のログを grep で追跡可能
- 障害時に「何が起きたか」がわかる

## 7. 今後の展望

### 現状の課題

- LLMは汎用的な知識のみ
- 社内情報・製品情報に答えられない
- 毎回同じ文脈を送り直している

## RAG（検索拡張生成）の適用

ユーザー質問



Vector Search  
(埋め込み検索)

← 関連ドキュメント検索



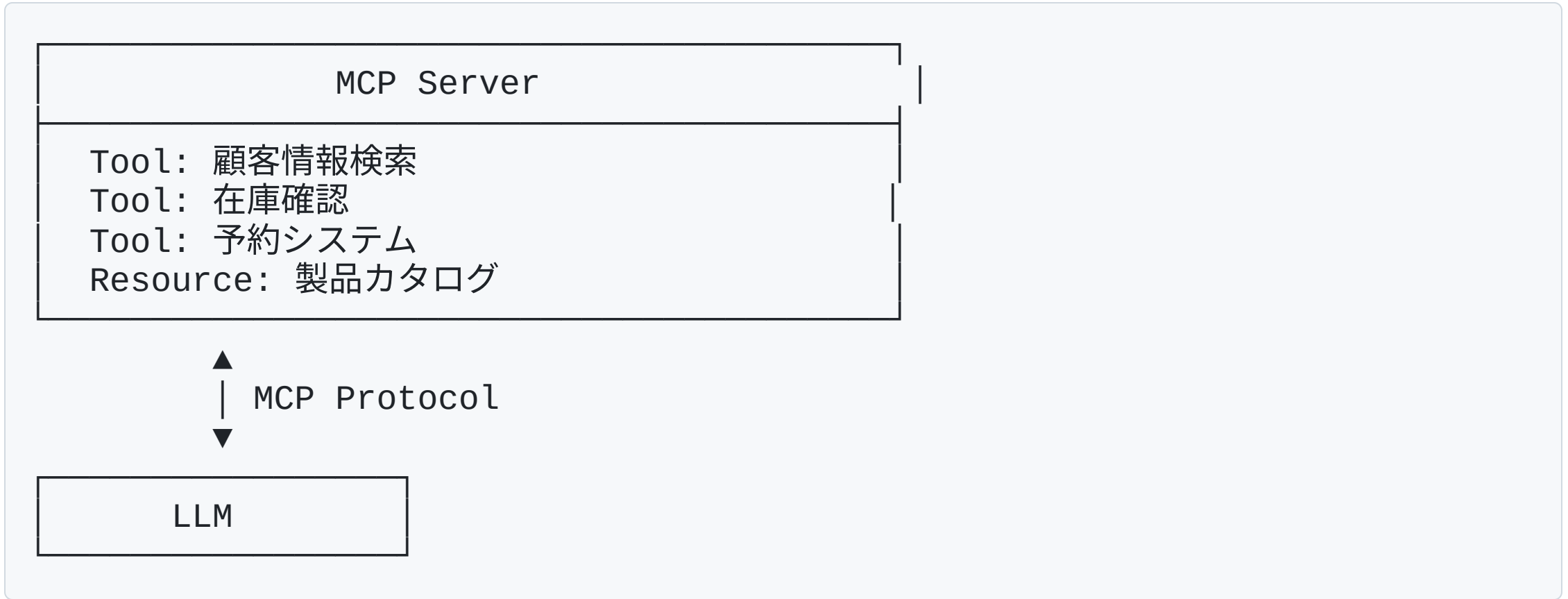
LLM

← 検索結果 + 質問 で回答生成

### 期待効果

- 社内FAQ、製品マニュアルを参照して回答
- 「最新の情報」に基づいた応答

## MCP (Model Context Protocol) の適用



# MCP で何が変わるか

## Before（現状）

- 天気API呼び出しをハードコード
- 新しい機能追加 = コード変更

## After（MCP適用後）

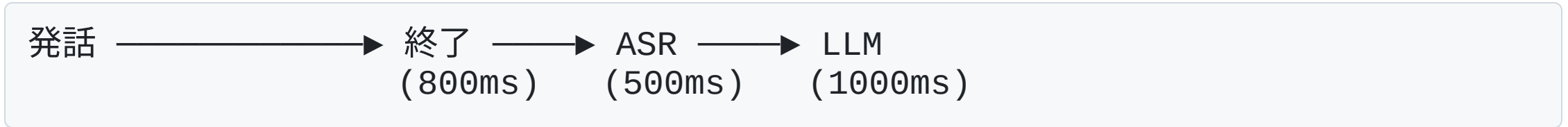
- ツールをMCPサーバーとして外部化
- LLMが必要に応じてツールを呼び出し
- 新機能 = MCPサーバー追加（コード変更不要）

## 例

```
{ "tool": "check_inventory",
 "params": { "product_id": "ABC123" } }
```

# ストリーミングASRへの進化

## 現状（バッチ方式）



## 将来（ストリーミング）



## 期待効果

- 体感レイテンシ大幅改善
- 「考え中」の沈黙が減る

# ロードマップ

| フェーズ | 内容                       |
|------|--------------------------|
| 現在   | バッチASR、インテント分類、フォールバック構成 |
| 次    | RAG導入（社内FAQ対応）           |
| その次  | MCP対応（ツール外部化）            |
| 将来   | ストリーミングASR、リアルタイム割り込み    |

# まとめ：技術選択の理由

| 選択           | 理由              |
|--------------|-----------------|
| Rust + Tokio | GCなし、軽量タスク、安全性  |
| 1セッション=1タスク  | ロック競合回避、障害分離    |
| イベント駆動       | 疎結合、テストビリティ     |
| インテント分類      | LLMコスト削減、処理の決定性 |
| 2段フォールバック    | 可用性担保           |

# 持ち帰り

## 1. データフローに沿って設計する

入力→処理→出力の流れを明確に

## 2. 全部LLMに任せない

インテント分類で振り分け、専用処理を活用

## 3. フォールバックで可用性担保

クラウド→ローカルの2段構成

## 4. 将来を見据えた拡張ポイント

RAG、MCP で「後から機能追加」できる設計

## Q&A

ご質問があればどうぞ

## 参考：モジュール構成

```
src/
├── transport/ # UDP/TCP ソケット
├── sip/ # SIP パース・トランザクション
├── rtp/ # RTP/RTCP、codec
├── session/ # 呼制御・状態管理
├── app/ # 対話ロジック
├── ai/ # ASR/LLM/TTS クライアント
│ └── intent.rs # インテント分類
├── media/ # 録音
├── http/ # REST/SSE
└── db/ # Tsurugi クライアント
```